



COURSE CODE/TITLE:	CPE 010	SECTION:	CPE21S1
DATE PERFORMED:	07-17-26	DATE SUBMITTED:	07-22-26
NAME:	Paula Esguerra	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 2 Arrays Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 2-1. Initial Driver Program:

Screenshot

```
main.cpp:2:1: warning: 'main' is deprecated [-Wdeprecated-declarations]
1  #include<iostream>
2  #include <string.h>
3
4  class Student{
5  private:
6      std::string studentName;
7      int studentAge;
8  public:
9      //constructor
10     Student(std::string newName = "John Doe", int newAge = 18) {
11         studentName = std::move(newName);
12         studentAge = newAge;
13         std::cout << "Constructor Called." << std::endl;
14     };
15
16     //destructor
17     ~Student (){
18         std::cout << "Destructor Called." << std::endl;
19     }
20
21     //Copy Constructor
22     Student (const Student &copyStudent) {
23         std::cout << "Copy Costructor Called" << std::endl;
24         studentName = copyStudent.studentName;
25         studentAge = copyStudent.studentAge;
26     }
27
28     //Display Attributes
29     void printDetails () {
30         std::cout << this->studentName << " " << this->studentAge << std::endl;
31     }
```

	<pre> 33 }; 34 35 int main (){ 36 Student student1("Roman", 28); 37 Student student2(student1); 38 Student student3; 39 student3 = student2; 40 41 return 0; 42 } </pre> PS C:\Users\TIPQC\Documents\CPE 010_Esguerra_CPE21S2\HOA # Constructor Called. Copy Costructor Called Constructor Called. Destructor Called. Destructor Called. Destructor Called.
Observation	Student 3 has student name "Roman" and student age "28" using new function, the student name will change to "John Doe" and its age as "18", using move function, it steals the internal pointer and converts an object into a rvalue reference, it so happens that instead "Roman" "28" is on student3, its returns to student1 then at student3, "John Doe" "18" replaces it, which the new functions do its job. So Constructor is called after "Roman" "28" is at student1 and "John Doe" "18" is replaced in student3. Then destructor was used three times, where each student in int main is deleted.
Table 2-2. Modified Driver Program with Student Lists :	

Screenshot

```

1  #include<iostream>
2  #include <string.h>
3
4  class Student{
5  private:
6      std::string studentName;
7      int studentAge;
8  public:
9      //constructor
10     Student(std::string newName = "John Doe", int newAge = 18) {
11         studentName = std::move(newName);
12         studentAge = newAge;
13         std::cout << "Constructor Called." << std::endl;
14     };
15
16     //destructor
17     ~Student () {
18         std::cout << "Destructor Called." << std::endl;
19     }
20
21     //Copy Constructor
22     Student (const Student &copyStudent) {
23         std::cout << "Copy Costructor Called" << std::endl;
24         studentName = copyStudent.studentName;
25         studentAge = copyStudent.studentAge;
26     }
27
28     //Display Attributes
29     void printDetails () {
30         std::cout << this->studentName << " " << this->studentAge << std::endl;
31     }
32
33 };
34
35 int main (){
36     const size_t j = 5;
37
38     Student studentList[j] = {};
39     std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40     int ageList[j] = {15, 16, 18, 19, 16};
41     return 0;
42 }

```

```

PS C:\Users\TIPQC\Documents\CPE 010_Esguerra_CPE21S2\HOA # 2> cd guerra_CPE21S2\HOA # 2\ ; if ($?) { g++ main.cpp -o main } ; if
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Documents\CPE 010_Esguerra_CPE21S2\HOA # 2>

```

Observation

John Doe was constructed 5 times, a line of 5 names were declared at the string list as also the ages that were given. Also destructor was initialized 5

times also.

Table 2-3. Final Driver Program

Screenshot: LOOP A

```

1  #include <iostream>
2  #include <string.h>
3
4  class Student{
5  private:
6      std::string studentName;
7      int studentAge;
8  public:
9      //constructor
10     Student(std::string newName = "John Doe", int newAge = 18) {
11         studentName = std::move(newName);
12         studentAge = newAge;
13         std::cout << "Constructor Called." << std::endl;
14     };
15
16     //destructor
17     ~Student (){}
18     std::cout << "Destructor Called." << std::endl;
19 }
20
21 //Copy Constructor
22 Student (const Student &copyStudent) {
23     std::cout << "Copy Costructor Called" << std::endl;
24     studentName = copyStudent.studentName;
25     studentAge = copyStudent.studentAge;
26 }
27
28 //Display Attributes
29 void printDetails () {
30     std::cout << this->studentName << " " << this->studentAge << std::endl;
31 }
32
33 };
34
35 int main (){
36     const size_t j = 5;

```

```

35 int main (){
36     const size_t j = 5;
37
38     Student studentList[j] = {};
39     std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40     int ageList[j] = {15, 16, 18, 19, 16};
41
42     for (int i = 0; i < j; i++) { //loop A
43         Student *ptr = new Student(namesList[i], ageList[i]);
44         studentList[i] = *ptr;
45     }
46
47
48     return 0;
49 }

```

Observation:	Constructor was called 5 times, using john doe, then the string is declared five names and their ages. Carly was used pointer to be constructed then t used point to const then put to the list and delete ptr cannot be used until main exits.
Screenshot: Loop B	<pre> 1 #include<iostream> 2 #include <string.h> 3 4 class Student{ 5 private: 6 std::string studentName; 7 int studentAge; 8 public: 9 //constructor 10 Student(std::string newName = "John Doe", int newAge = 18) { 11 studentName = std::move(newName); 12 studentAge = newAge; 13 std::cout << "Constructor Called." << std::endl; 14 }; 15 16 //destructor 17 ~Student (){} 18 std::cout << "Destructor Called." << std::endl; 19 } 20 21 //Copy Constructor 22 Student (const Student &copyStudent) { 23 std::cout << "Copy Costructor Called" << std::endl; 24 studentName = copyStudent.studentName; 25 studentAge = copyStudent.studentAge; 26 } 27 28 //Display Attributes 29 void printDetails () { 30 std::cout << this->studentName << " " << this->studentAge << std::endl; 31 } 32 33 }; 34 35 int main (){ 36 const size_t j = 5; 37 38 Student studentList[j] = {}; 39 std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"}; 40 int ageList[j] = {15, 16, 18, 19, 16}; 41 42 for (int i = 0; i < j; i++) { //loop B 43 studentList[i].printDetails(); 44 } 45 46 return 0; 47 } </pre>

Observation:	John doe was constructed by 5 times, then it used pointer then printed by five times, then it was deleted
Output: LOOP A (first) LOOP B (second)	<pre> Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. John Doe 18 John Doe 18 John Doe 18 John Doe 18 John Doe 18 Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. </pre>
Observation	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ACTIVITY NO. 2 Arrays Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 2-1. Initial Driver Program:

Screenshot

```
main.cpp > g++ main.cpp
1  #include<iostream>
2  #include <string.h>
3
4  class Student{
5  private:
6      std::string studentName;
7      int studentAge;
8  public:
9      //constructor
10     Student(std::string newName = "John Doe", int newAge = 18) {
11         studentName = std::move(newName);
12         studentAge = newAge;
13         std::cout << "Constructor Called." << std::endl;
14     };
15
16     //destructor
17     ~Student (){
18         std::cout << "Destructor Called." << std::endl;
19     }
20
21     //Copy Constructor
22     Student (const Student &copyStudent) {
23         std::cout << "Copy Costructor Called" << std::endl;
24         studentName = copyStudent.studentName;
25         studentAge = copyStudent.studentAge;
26     }
27
28     //Display Attributes
29     void printDetails () {
30         std::cout << this->studentName << " " << this->studentAge << std::endl;
31     }
}
```

	<pre> 33 }; 34 35 int main (){ 36 Student student1("Roman", 28); 37 Student student2(student1); 38 Student student3; 39 student3 = student2; 40 41 return 0; 42 } </pre> PS C:\Users\TIPQC\Documents\CPE 010_Esguerra_CPE21S2\HOA # Constructor Called. Copy Costructor Called Constructor Called. Destructor Called. Destructor Called. Destructor Called.
Observation	Student 3 has student name "Roman" and student age "28" using new function, the student name will change to "John Doe" and its age as "18", using move function, it steals the internal pointer and converts an object into a rvalue reference, it so happens that instead "Roman" "28" is on student3, its returns to student1 then at student3, "John Doe" "18" replaces it, which the new functions do its job. So Constructor is called after "Roman" "28" is at student1 and "John Doe" "18" is replaced in student3. Then destructor was used three times, where each student in int main is deleted.
Table 2-2. Modified Driver Program with Student Lists :	

Screenshot

```

1  #include<iostream>
2  #include <string.h>
3
4  class Student{
5  private:
6      std::string studentName;
7      int studentAge;
8  public:
9      //constructor
10     Student(std::string newName = "John Doe", int newAge = 18) {
11         studentName = std::move(newName);
12         studentAge = newAge;
13         std::cout << "Constructor Called." << std::endl;
14     };
15
16     //destructor
17     ~Student () {
18         std::cout << "Destructor Called." << std::endl;
19     }
20
21     //Copy Constructor
22     Student (const Student &copyStudent) {
23         std::cout << "Copy Costructor Called" << std::endl;
24         studentName = copyStudent.studentName;
25         studentAge = copyStudent.studentAge;
26     }
27
28     //Display Attributes
29     void printDetails () {
30         std::cout << this->studentName << " " << this->studentAge << std::endl;
31     }

```

```

32
33 };
34
35 int main (){
36     const size_t j = 5;
37
38     Student studentList[j] = {};
39     std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40     int ageList[j] = {15, 16, 18, 19, 16};
41     return 0;
42 }

```

```

PS C:\Users\TIPQC\Documents\CPE 010_Esguerra_CPE21S2\HOA # 2> cd guerra_CPE21S2\HOA # 2\ ; if ($?) { g++ main.cpp -o main } ; if
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Documents\CPE 010_Esguerra_CPE21S2\HOA # 2>

```

Observation

John Doe was constructed 5 times, a line of 5 names were declared at the string list as also the ages that were given. Also destructor was initialized 5

times also.

Table 2-3. Final Driver Program

Screenshot: LOOP A

```

1  #include <iostream>
2  #include <string.h>
3
4  class Student{
5  private:
6      std::string studentName;
7      int studentAge;
8  public:
9      //constructor
10     Student(std::string newName = "John Doe", int newAge = 18) {
11         studentName = std::move(newName);
12         studentAge = newAge;
13         std::cout << "Constructor Called." << std::endl;
14     };
15
16     //destructor
17     ~Student (){}
18     std::cout << "Destructor Called." << std::endl;
19 }
20
21 //Copy Constructor
22 Student (const Student &copyStudent) {
23     std::cout << "Copy Costructor Called" << std::endl;
24     studentName = copyStudent.studentName;
25     studentAge = copyStudent.studentAge;
26 }
27
28 //Display Attributes
29 void printDetails () {
30     std::cout << this->studentName << " " << this->studentAge << std::endl;
31 }
32
33 };
34
35 int main (){
36     const size_t j = 5;

```

```

35 int main (){
36     const size_t j = 5;
37
38     Student studentList[j] = {};
39     std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40     int ageList[j] = {15, 16, 18, 19, 16};
41
42     for (int i = 0; i < j; i++) { //loop A
43         Student *ptr = new Student(namesList[i], ageList[i]);
44         studentList[i] = *ptr;
45     }
46
47
48     return 0;
49 }

```

Observation:	Constructor was called 5 times, using john doe, then the string is declared five names and their ages. Carly was used pointer to be constructed then t used point to const then put to the list and delete ptr cannot be used until main exits.
Screenshot: Loop B	<pre> 1 #include<iostream> 2 #include <string.h> 3 4 class Student{ 5 private: 6 std::string studentName; 7 int studentAge; 8 public: 9 //constructor 10 Student(std::string newName = "John Doe", int newAge = 18) { 11 studentName = std::move(newName); 12 studentAge = newAge; 13 std::cout << "Constructor Called." << std::endl; 14 }; 15 16 //destructor 17 ~Student (){} 18 std::cout << "Destructor Called." << std::endl; 19 } 20 21 //Copy Constructor 22 Student (const Student &copyStudent) { 23 std::cout << "Copy Costructor Called" << std::endl; 24 studentName = copyStudent.studentName; 25 studentAge = copyStudent.studentAge; 26 } 27 28 //Display Attributes 29 void printDetails () { 30 std::cout << this->studentName << " " << this->studentAge << std::endl; 31 } 32 33 }; 34 35 int main (){ 36 const size_t j = 5; 37 38 Student studentList[j] = {}; 39 std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"}; 40 int ageList[j] = {15, 16, 18, 19, 16}; 41 42 for (int i = 0; i < j; i++) { //loop B 43 studentList[i].printDetails(); 44 } 45 46 return 0; 47 } </pre>

Observation:	John doe was constructed by 5 times, then it used pointer then printed by five times, then it was deleted
Output: LOOP A (first) LOOP B (second)	<pre> Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. John Doe 18 John Doe 18 John Doe 18 John Doe 18 John Doe 18 Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. </pre>
Observation	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

Problem 5: Discuss the

```

1  #include<iostream>
2  #include <string>
3
4  class GroceryItem {
5      private:
6          std::string name;
7          int price;
8          int quantity;
9      public:
10         virtual ~GroceryItem() {}
11         virtual void display() = 0;
12         virtual int calculateCost() = 0;
13     };
14
15     class Fruit : public GroceryItem{
16     private:
17         std::string fruitName;
18         int fruit_price;
19         int Fquantity;
20
21     public:
22         //constructor
23         Fruit( std::string n, int p, int q ){
24             fruitName = n;
25             fruit_price = p;
26             Fquantity = q;
27             std::cout << "Constructor Called. Fruit: " << fruitName << std::endl;
28         }
29     }
30

```

In this part of the code, I added an additional class, Grocery, because I was having trouble with how to print Fruit and Vegetable in the last part. As you can see in the snippet, I used Virtual in lines 10 to 12. The other alternative was suggesting I change some major parts in the code; it was a lot to change, so I decided to stick with what I've searched for in Gemini. As I can understand, virtual in class enables runtime polymorphism; it ensures that I could execute a function as an overridden version in the class. So in some part of the code, instead of using const, I changed it to override. Before I created groceryitem, I started by fixing Class Fruit and Vegetable, using the reference of the activity before the supplementary and revised it using some links from the internet and some questions in gemini.

```

31 //destructor
32 ~Fruit () {
33     std::cout << "Destructor Called. What was Deleted:" << fruitName << std::endl;
34 }
35
36 //Copyconstructor
37 Fruit (const Fruit &copyFruit) {
38     fruitName = copyFruit.fruitName;
39     fruit_price = copyFruit.fruit_price;
40     Fquantity = copyFruit.Fquantity;
41     std::cout << "Copy Constructor Called. Copied: " << fruitName << std::endl;
42 }
43
44 //overwrites an existing object - copy assignment
45 Fruit& operator=(const Fruit &copyFruit ) {
46     std::cout << "Copy Assignment Called. Assigning: " << copyFruit.fruitName << "\n\n";
47     if (this != &copyFruit) {
48         fruitName = copyFruit.fruitName;
49         fruit_price = copyFruit.fruit_price;
50         Fquantity = copyFruit.Fquantity;
51     }
52     return *this;
53 }
54
55 int calculateCost() override{
56     return fruit_price*Fquantity;
57 }
58
59 void display() override{
60     std::cout << "\nWhat Fruit: " << fruitName << " Price: PHP " << fruit_price << std::endl;
61     std::cout << "How many?: " << Fquantity << "\n";
62     std::cout << "Total: PHP" << calculateCost() << std::endl << std::endl;
63 }

```

Lines 31 - 63. Deconstruction was easy; at first I just typed and copied the activity before the supplementary, then I looked for some ideas on the internet. It was actually possible to print what was deleted to know what it was. I knew it was possible, but I had no idea how. Then, moving on to the copy constructor, this was easy too; it was easy to understand, but I had some trouble printing it since I had little practice understanding it, so at the copy assignment part, it first returns a reference to the object and then accepts the object by reference to avoid costly copy when passing it to the function. The const is to ensure it won't modify the source object. Line 47 is to check if the object is being assigned; then it points to the current object on the left side of the "=", so if they share the same address, it skips the function.

```

64 };
65
66
67 //class vegetables
68 class Vegetables : public GroceryItem{
69     private:
70         std::string vegName;
71         int veg_price;
72         int Vquantity;
73
74     public:
75     //constructor
76     Vegetables( std::string n, int p, int q ){
77         vegName = n;
78         veg_price = p;
79         Vquantity = q;
80         std::cout << "Constructor Called. Vegetables: " << vegName << std::endl;
81     }
82
83     //deconstructor
84     ~Vegetables () {
85         std::cout << "Desconstructor Called. What was Deleted: " << vegName << std::endl;
86     }
87
88     //Copyconstructor
89     Vegetables (const Vegetables &copyVegetables) {
90         std::cout << "Copy Constructor Called. Copied: " << vegName << std::endl;
91         vegName = copyVegetables.vegName;
92         veg_price = copyVegetables.veg_price;
93         Vquantity = copyVegetables.Vquantity;
94     }
95
96 //overwrites an existing object - copy assignment
97 Vegetables& operator=(const Vegetables &copyVegetables ) {
98     std::cout << "Copy Assignment Called. Assigning: " << copyVegetables.vegName << "\n\n";
99     if (this != &copyVegetables) {
100         vegName = copyVegetables.vegName;
101         veg_price = copyVegetables.veg_price;
102         Vquantity = copyVegetables.Vquantity;
103     }
104     return *this;
105 }
106
107 int calculateCost() override{
108     return veg_price*Vquantity;
109 }
110
111 void display() override{
112     std::cout << "\nWhat Vegetables: " << vegName << " Price: PHP" << veg_price << std::endl;
113     std::cout << "How many?: " << Vquantity << std::endl;
114     std::cout << "Total: PHP" << calculateCost() << "\n\n";
115 }
116
117 };
118
119
120

```

Line 68 to 199 is completely copied and pasted from Fruits, since for my opinion I don't have to change anything but the same, since they will have the same function that I will be needing to have. So this concludes that I have finished the Class, and now I will explain the int main functions.

```

120 int main () {
121     GroceryItem* list[10];
122
123     list[0] = new Fruit("Orange", 10, 1);
124     list[1] = new Fruit("Apple", 150, 7);
125     list[2] = new Fruit("Peach", 500, 20);
126     list[3] = new Fruit("Lychee", 100, 30);
127     list[4] = new Fruit("Banana", 250, 20);
128     list[5] = new Vegetables ("Potato", 10, 1);
129     list[6] = new Vegetables ("Kangkong", 10, 1);
130     list[7] = new Vegetables ("Sitaw", 25, 1);
131     list[8] = new Vegetables ("Gabi", 90, 3);
132     list[9] = new Vegetables ("Carrots", 100, 9);
133
134     for (int i = 0; i < 10; i++) {
135         if(list[i] != nullptr) {
136             list[i] ->display();
137         }
138     }
139
140     int totalSum = 0;
141     for (int i = 0; i < 10; i++){
142         if (list[i] != nullptr) {
143             totalSum += list[i] ->calculateCost();
144         }
145     }
146
147     std::cout << "\n\nThe Whole Total: PHP " << totalSum << "\n\n\n";
148 }

```

In items of 10 kinds, I listed the fruits and vegetables, and because I used class Grocery I was able to put together the two classes in one list too. Lines 134 - 138 ensure the code won't crash if anGrocery,ot is empty; the display is virtual from the first snippet; it resolves the function call at runtime; and it automatically runs for items fruits 0-4 and vegetables 5-9.

```
148
149     Fruit originalFruit("Orange", 10, 1);
150     Fruit emptyFruit("Placeholder", 45, 5);
151     emptyFruit = originalFruit;
152
153     Vegetables originalVegetables("Gabi", 90, 3);
154     Vegetables emptyVegetables("Placeholder", 85, 5);
155     emptyVegetables = originalVegetables;
156
157     delete list[5];
158
159     list[5] = nullptr;
160     delete list[9];
161     delete list[8];
162     delete list[7];
163     delete list[6];
164     delete list[5];
165     delete list[4];
166     delete list[3];
167     delete list[2];
168     delete list[1];
169     delete list[0];
170
171     return 0;
172
173
174 }
```

And lastly, I accessed copy constructor and copy assignment, also destructor. The existing data inside emptyfruit is completely overwritten by the values of the original fruit. Deletes the heap memory by the Potato object, this is at line 157-159, at index 5, then setting the list to clear out the dead address so the pointer will not point to an unallocated memory. Then delete ten times in reverse order.

OUTPUT:

```
C:\Users\Paramount Laptop 5  ×  +  ∨  
Constructor Called. Fruit: Orange  
Constructor Called. Fruit: Apple  
Constructor Called. Fruit: Peach  
Constructor Called. Fruit: Lychee  
Constructor Called. Fruit: Banana  
Constructor Called. Vegetables: Potato  
Constructor Called. Vegetables: Kangkong  
Constructor Called. Vegetables: Sitaw  
Constructor Called. Vegetables: Gabi  
Constructor Called. Vegetables: Carrots  
  
What Fruit: Orange  Price: PHP 10  
How many?: 1  
Total: PHP10  
  
What Fruit: Apple  Price: PHP 150  
How many?: 7  
Total: PHP1050  
  
What Fruit: Peach  Price: PHP 500  
How many?: 20  
Total: PHP10000  
  
What Fruit: Lychee  Price: PHP 100  
How many?: 30  
Total: PHP3000  
  
What Fruit: Banana  Price: PHP 250  
How many?: 20  
Total: PHP5000
```



C:\Users\Paramount Laptop 5



```
Constructor Called. Fruit: Orange
Constructor Called. Fruit: Apple
Constructor Called. Fruit: Peach
Constructor Called. Fruit: Lychee
Constructor Called. Fruit: Banana
Constructor Called. Vegetables: Potato
Constructor Called. Vegetables: Kangkong
Constructor Called. Vegetables: Sitaw
Constructor Called. Vegetables: Gabi
Constructor Called. Vegetables: Carrots
```

```
What Fruit: Orange   Price: PHP 10
How many?: 1
Total: PHP10
```

```
What Fruit: Apple   Price: PHP 150
How many?: 7
Total: PHP1050
```

```
What Fruit: Peach   Price: PHP 500
How many?: 20
Total: PHP10000
```

```
What Fruit: Lychee   Price: PHP 100
How many?: 30
Total: PHP3000
```

```
What Fruit: Banana  Price: PHP 250  
How many?: 20  
Total: PHP5000
```

```
What Vegetables: Potato  Price: PHP10  
How many?: 1  
Total: PHP10
```

```
What Vegetables: Kangkong  Price: PHP10  
How many?: 1  
Total: PHP10
```

```
What Vegetables: Sitaw  Price: PHP25  
How many?: 1  
Total: PHP25
```

```
What Vegetables: Gabi  Price: PHP90  
How many?: 3  
Total: PHP270
```

```
What Vegetables: Carrots  Price: PHP100
How many?: 9
Total: PHP900
```

```
The Whole Total: PHP 20275
```

```
Constructor Called. Fruit: Orange
Constructor Called. Fruit: Placeholder
Copy Assignment Called. Assigning: Orange
```

```
Constructor Called. Vegetables: Gabi
Constructor Called. Vegetables: Placeholder
Copy Assignment Called. Assigning: Gabi
```

```
Destructor Called. What was Deleted: Potato
Destructor Called. What was Deleted: Carrots
Destructor Called. What was Deleted: Gabi
Destructor Called. What was Deleted: Sitaw
Destructor Called. What was Deleted: Kangkong
Destructor Called. What was Deleted:Banana
Destructor Called. What was Deleted:Lychee
Destructor Called. What was Deleted:Peach
Destructor Called. What was Deleted:Apple
Destructor Called. What was Deleted:Orange
Destructor Called. What was Deleted: Gabi
Destructor Called. What was Deleted: Gabi
Destructor Called. What was Deleted:Orange
Destructor Called. What was Deleted:Orange
```

```
-----
Process exited after 0.1979 seconds with return value 0
Press any key to continue . . .
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Idea Generation, explanation, and revising.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: My original work is there, I asked how not give me the answers, I asked for examples too, so some part may be similar but I typed it all and understand almost all the concepts.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.